UNIVERSIDAD POLITÉCNICA DE PACHUCA

Ingeniería en Tecnologías de la Información e Innovación Digital

DESARROLLO DE APLICACIONES MÓVILES

Reporte de Práctica

Implementación de Column y LazyColumn en Jetpack Compose

Alumno: Daniel Jimenez Pluma

Grupo: TIID 04 01

Docente: Miguel Angel Montoya Cerro

Lugar: Zempoala, Hidalgo, México

Fecha: 2026

Índice general

1 Introducción **5**

1.1 Objetivo general 5

1.2 Objetivos específicos 5

1.3 Justificación 6

1.4 Estructura del documento 6

2 Marco teórico **6**

2.1 Android Studio 6

2.2 Kotlin 6

2.3 Jetpack Compose 7

2.4 Column 7

2.5 LazyColumn 7

2.6 Modifier 7

2.7 Text 7

2.8 Image 8

3 Desarrollo **8**

3.1 Creación del proyecto 8

3.2 Configuración de la actividad principal 9

3.3 Creación de la función composable 9

3.4 Organización de los elementos 10

3.5 Incorporación de la imagen 11

3.6 Implementación de los textos 11

3.7 Problema identificado 12

3.8 Implementación de LazyColumn 13

4 Análisis y argumentación **13**

5 Resultados **14**

6 Explicación del código **14**

6.1 Código completo 14

7 Elementos principales utilizados **17**

8 Conclusiones **17**

9 Trabajo futuro **18**

10 Anexos **18**

10.1 Evidencias adicionales 18

11 Referencias **19**

Índice de ilustraciones

1 Creación y configuración inicial del proyecto. 8
2 Configuración de la actividad principal. 9
3 Implementación de la función composable. 10
4 Organización vertical mediante Column. 10
5 Incorporación de la imagen en la interfaz. 11
6 Implementación y configuración de los componentes Text. 12
7 Problema de contenido fuera del área visible. 12
8 Resultado final utilizando LazyColumn. 13

Índice de tablas

1 Elementos principales utilizados en la práctica 17

1. Introducción

El desarrollo de aplicaciones móviles requiere herramientas que permitan construir interfaces de usuario de manera organizada, funcional y adaptable a diferentes tamaños de pantalla. Dentro del desarrollo de aplicaciones Android, Jetpack Compose proporciona un enfoque declarativo para la creación de interfaces mediante el lenguaje Kotlin.

La presente práctica se enfoca en la implementación y utilización de los componentes `Column` y `LazyColumn`, los cuales permiten organizar elementos de una interfaz de manera vertical. Durante el desarrollo se trabajó con diferentes componentes visuales, modificadores, textos e imágenes para construir una interfaz sencilla.

El problema identificado durante la práctica se relaciona con la cantidad de contenido que puede mostrarse dentro del espacio disponible de una pantalla. Mientras que `Column` permite organizar elementos verticalmente, cuando el contenido supera el espacio visible es necesario contar con un mecanismo de desplazamiento. Para resolver esta situación se implementó `LazyColumn`.

1.1. Objetivo general

Implementar una interfaz de usuario mediante Jetpack Compose utilizando los componentes `Column` y `LazyColumn`, comprendiendo sus características y diferencias para organizar contenido vertical dentro de una aplicación Android.

1.2. Objetivos específicos

- Crear un proyecto Android utilizando Android Studio, Kotlin y Jetpack Compose.
- Implementar una interfaz mediante funciones composables.
- Utilizar `Column` para organizar elementos verticalmente.
- Aplicar modificadores para controlar el tamaño y apariencia de los componentes.
- Incorporar una imagen mediante el componente `Image`.
- Implementar diferentes componentes `Text`.
- Identificar la limitación de `Column` cuando el contenido supera el espacio disponible.
- Sustituir `Column` por `LazyColumn` para permitir el desplazamiento vertical.

1.3. Justificación

La práctica resulta relevante debido a que permite comprender conceptos fundamentales para el desarrollo de interfaces Android mediante Jetpack Compose. El conocimiento de los componentes `Column` y `LazyColumn` permite seleccionar la estructura adecuada dependiendo de la cantidad y comportamiento del contenido que se desea mostrar.

Además, el ejercicio permite relacionar los conceptos teóricos con una implementación funcional, observando directamente el efecto que tienen los modificadores y componentes de Compose sobre la interfaz.

1.4. Estructura del documento

El documento se encuentra organizado en diferentes secciones. Primero se presenta el marco teórico relacionado con Android Studio, Kotlin y Jetpack Compose. Posteriormente se describe el desarrollo de la práctica y se presentan las evidencias obtenidas. Finalmente se incluye el análisis de resultados, las conclusiones, el trabajo futuro, los anexos y las referencias.

2. Marco teórico

2.1. Android Studio

Android Studio es el entorno de desarrollo utilizado para la creación de aplicaciones Android. Permite crear proyectos, editar código, administrar recursos, compilar aplicaciones y realizar pruebas mediante emuladores o dispositivos físicos.

En esta práctica fue utilizado como herramienta principal para crear el proyecto, implementar el código Kotlin y visualizar la interfaz desarrollada mediante Jetpack Compose.

2.2. Kotlin

Kotlin es el lenguaje de programación utilizado para desarrollar la aplicación. Es compatible con el desarrollo Android y permite utilizar Jetpack Compose mediante funciones declarativas.

El código desarrollado durante la práctica está escrito en Kotlin y contiene la actividad principal junto con la función composable encargada de construir la interfaz.

2.3. Jetpack Compose

Jetpack Compose es un toolkit moderno para la construcción de interfaces de usuario en Android mediante un enfoque declarativo. La interfaz se define mediante funciones composables escritas en Kotlin.

En lugar de construir toda la interfaz mediante archivos XML, Compose permite describir los componentes directamente mediante código.

2.4. Column

El componente `Column` permite organizar diferentes elementos de manera vertical. Los componentes contenidos dentro de este elemento se colocan uno debajo de otro.

Esta estructura es adecuada para interfaces cuyo contenido puede visualizarse dentro del espacio disponible en pantalla.

2.5. LazyColumn

El componente `LazyColumn` permite organizar elementos verticalmente y proporciona desplazamiento cuando el contenido supera el espacio disponible.

Los elementos pueden agregarse mediante bloques `item{}`, como se realizó en la práctica.

2.6. Modifier

Los modificadores permiten controlar diferentes características de los componentes de Jetpack Compose.

Durante la práctica se utilizaron principalmente los siguientes:

- `fillMaxSize()`
- `fillMaxWidth()`
- `height()`
- `background()`

2.7. Text

El componente `Text` permite mostrar texto dentro de una interfaz. Puede configurarse mediante diferentes propiedades, como tamaño de fuente, color y alineación.

2.8. Image

El componente `Image` permite mostrar imágenes dentro de la interfaz. Para cargar el recurso gráfico utilizado en esta práctica se empleó `painterResource()`.

3. Desarrollo

3.1. Creación del proyecto

El desarrollo comenzó con la creación de un nuevo proyecto en Android Studio utilizando Kotlin y Jetpack Compose.

Se utilizó una actividad basada en Compose para construir la interfaz de usuario de la aplicación.

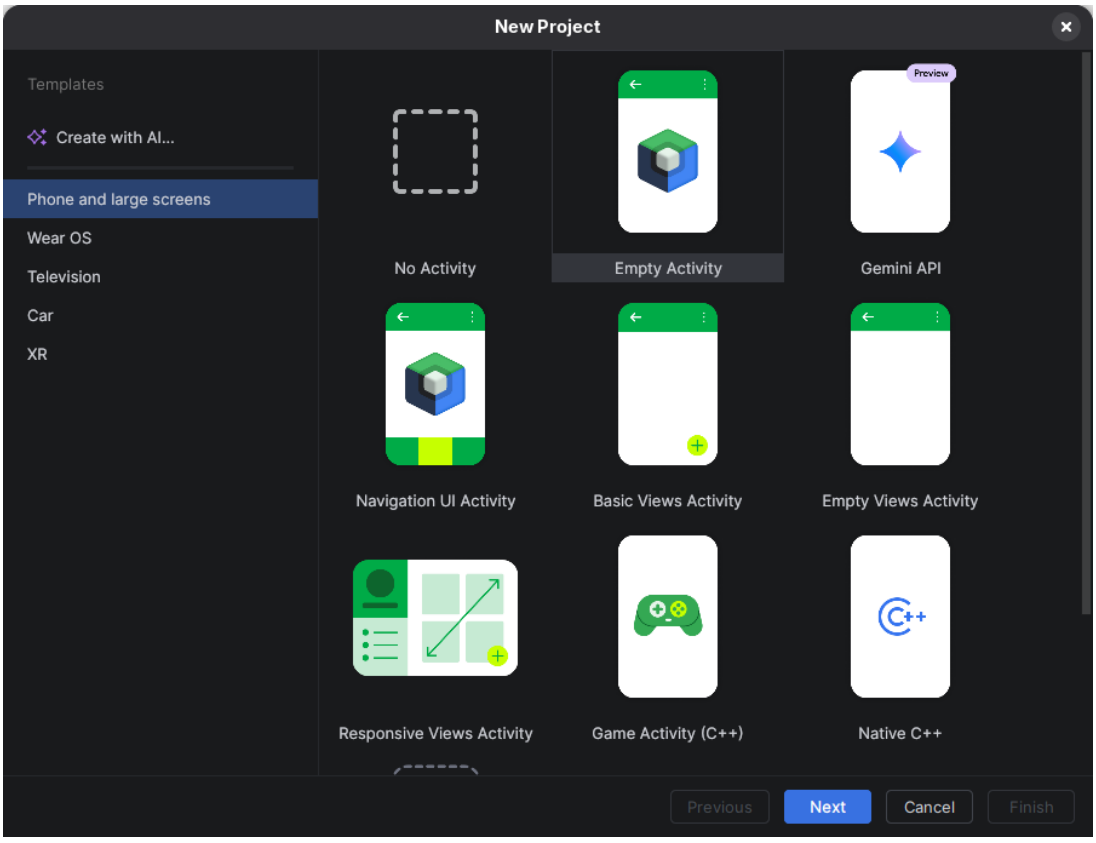


Ilustración 1: Creación y configuración inicial del proyecto.

La primera evidencia muestra la configuración inicial del proyecto utilizado para desarrollar la práctica.

3.2. Configuración de la actividad principal

Después de crear el proyecto se trabajó sobre la actividad principal. En ella se conserva el método `setContent`, encargado de establecer la interfaz construida mediante Jetpack Compose.

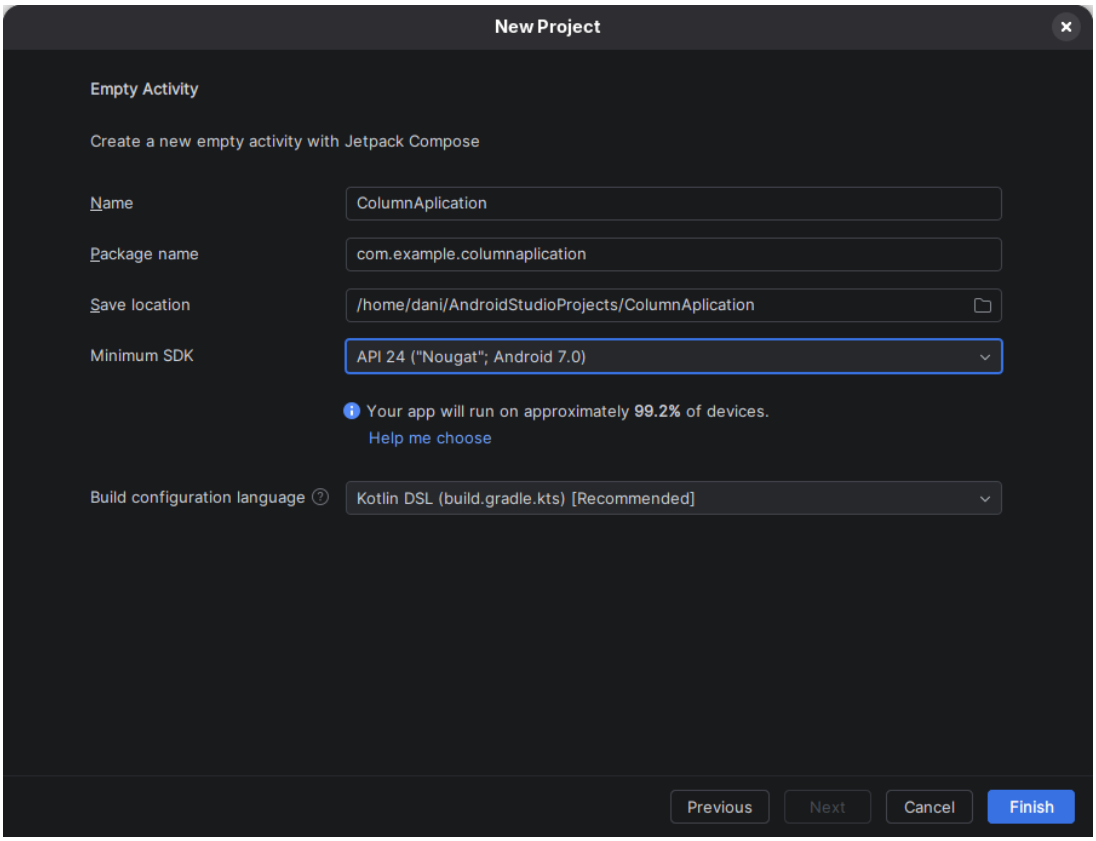


Ilustración 2: Configuración de la actividad principal.

La segunda evidencia muestra el entorno de desarrollo utilizado para continuar con la implementación de la interfaz.

3.3. Creación de la función composable

Se creó la función `app()`, identificada mediante la anotación `@Composable`. Esta función contiene los elementos que forman la interfaz.

También se utilizó la anotación `@Preview` para visualizar la interfaz desde Android Studio.

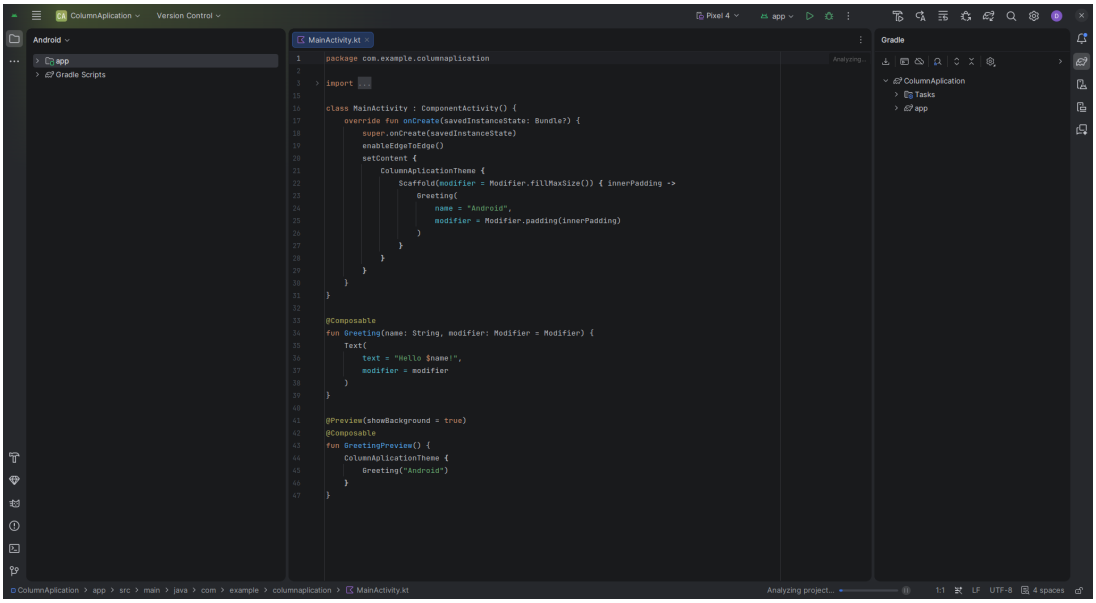


Ilustración 3: Implementación de la función composable.

3.4. Organización de los elementos

Inicialmente se trabajó con una estructura vertical mediante `Column`. Esto permitió colocar los diferentes componentes uno debajo de otro.

La organización vertical resulta útil para evitar que los elementos se superpongan y permite establecer una estructura clara dentro de la interfaz.

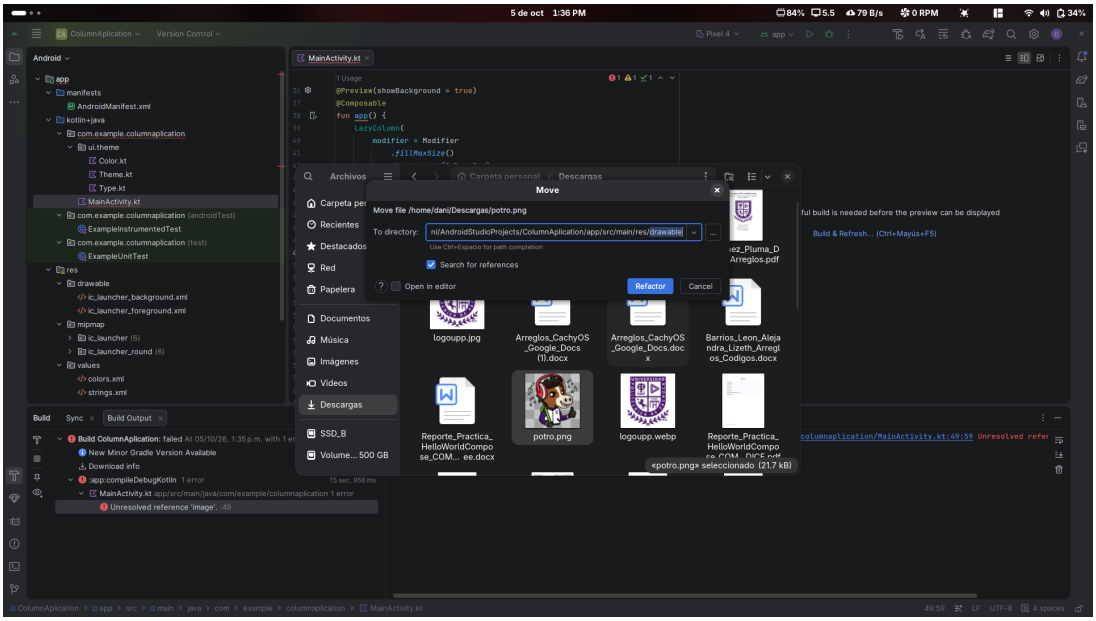


Ilustración 4: Organización vertical mediante Column.

3.5. Incorporación de la imagen

Se incorporó una imagen mediante el componente `Image`. El recurso utilizado se encuentra dentro de los recursos del proyecto y se obtiene mediante `painterResource`.

La imagen fue configurada con un ancho máximo y una altura de 400 dp.

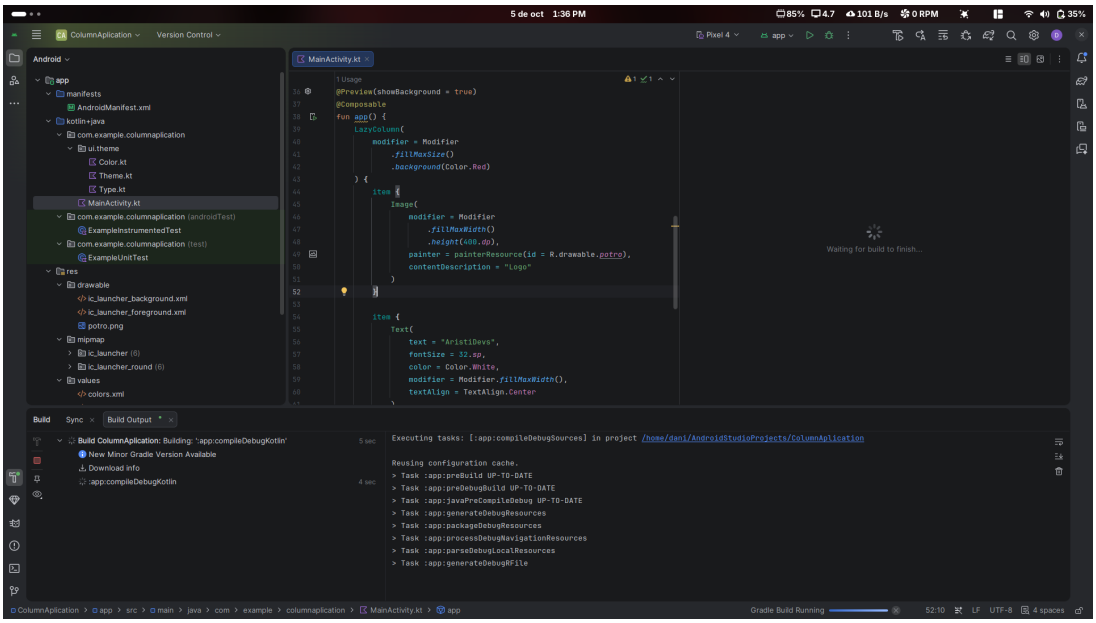


Ilustración 5: Incorporación de la imagen en la interfaz.

3.6. Implementación de los textos

Posteriormente se agregaron diferentes componentes `Text` para mostrar información dentro de la interfaz.

El texto principal “AristiDevs” fue configurado con un tamaño de fuente de 32 sp, color blanco y alineación centrada. También se agregaron los textos “Suscribete” y “Hola”.

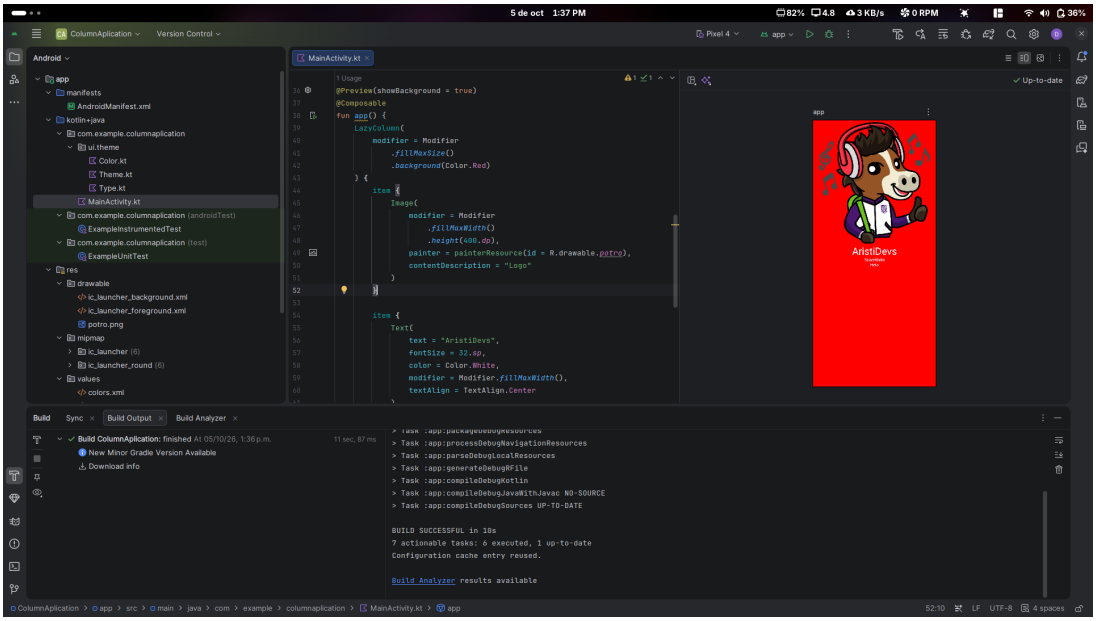


Ilustración 6: Implementación y configuración de los componentes Text.

3.7. Problema identificado

Durante las pruebas se observó que el contenido podía superar el espacio visible de la pantalla. Debido a esto, los elementos ubicados en la parte inferior podían quedar fuera del área visible.

Esta situación permitió identificar una de las principales limitaciones de una organización basada únicamente en `Column`: el contenido no cuenta automáticamente con desplazamiento vertical.

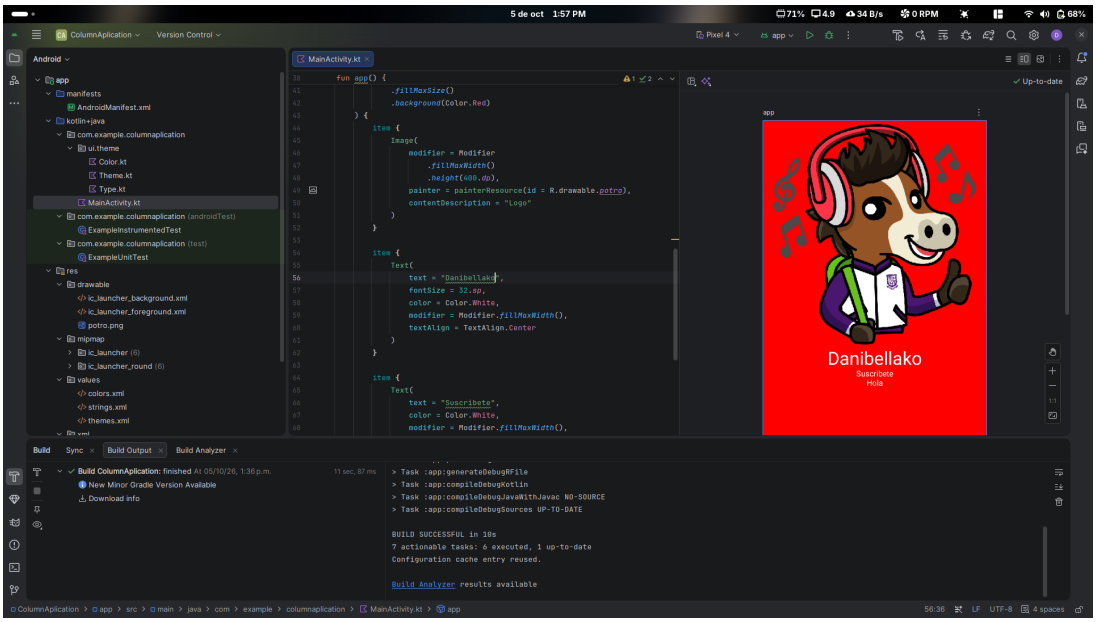


Ilustración 7: Problema de contenido fuera del área visible.

3.8. Implementación de LazyColumn

Para solucionar el problema identificado se sustituyó el contenedor `Column` por `LazyColumn`.

Los diferentes elementos se colocaron dentro de bloques `item{}`, permitiendo que formen parte del contenido desplazable de la interfaz.

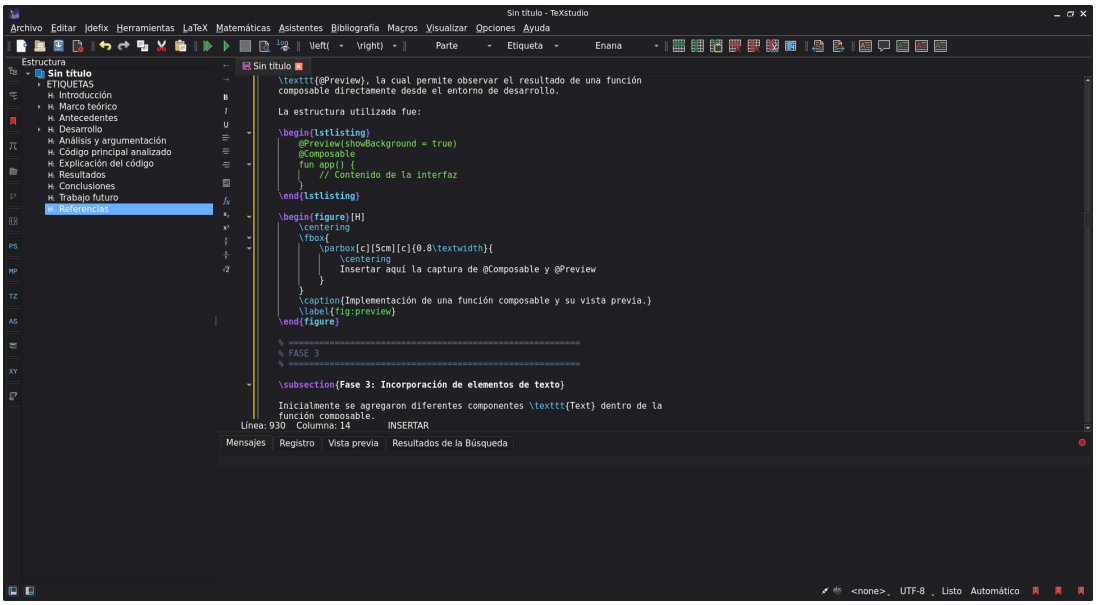


Ilustración 8: Resultado final utilizando LazyColumn.

La última evidencia muestra el resultado final de la aplicación después de implementar `LazyColumn`.

4. Análisis y argumentación

El desarrollo de la práctica permitió observar directamente las diferencias entre `Column` y `LazyColumn`.

En primera instancia, `Column` permitió organizar los elementos verticalmente de manera sencilla. Sin embargo, cuando el contenido supera el espacio disponible de la pantalla, esta estructura no proporciona por sí misma el desplazamiento necesario.

La implementación de `LazyColumn` permitió solucionar esta situación. Al utilizar bloques `item{}`, los elementos pasaron a formar parte de una estructura que permite desplazarse verticalmente.

También se comprobó la importancia de los modificadores. Por ejemplo, `fillMaxSize()` permite utilizar el espacio disponible, `fillMaxWidth()` permite utilizar el ancho disponible y `height()` permite establecer una dimensión específica.

La combinación de estos componentes permitió obtener una interfaz funcional y organizada.

5. Resultados

Como resultado de la práctica se obtuvo una aplicación Android desarrollada con Kotlin y Jetpack Compose.

La interfaz final contiene una imagen, tres elementos de texto y un fondo de color rojo. Los componentes se encuentran organizados verticalmente mediante `LazyColumn`.

También se comprobó el funcionamiento de los modificadores empleados para controlar el tamaño, ancho, altura, fondo y alineación de los componentes.

El resultado obtenido permitió cumplir con el objetivo de comprender la diferencia entre una estructura vertical convencional y una estructura que permite desplazamiento.

6. Explicación del código

La actividad principal se encuentra definida mediante la clase `MainActivity`, que hereda de `ComponentActivity`.

Dentro del método `onCreate()` se utiliza `setContent` para establecer la interfaz desarrollada mediante Compose.

La función `app()` está identificada con `@Composable` y contiene la interfaz principal.

El componente `LazyColumn` se utiliza como contenedor principal y recibe los modificadores `fillMaxSize()` y `background(Color.Red)`.

La imagen se muestra mediante `Image`, utilizando `painterResource(id = R.drawable.potro)` para obtener el recurso.

Los componentes `Text` permiten mostrar los textos de la interfaz. El primer texto utiliza un tamaño de 32 sp y todos utilizan color blanco y alineación centrada.

6.1. Código completo

```
1  package com.example.columnaplication
2
3  import android.os.Bundle
4  import androidx.activity.ComponentActivity
5  import androidx.activity.compose.setContent
6  import androidx.activity.enableEdgeToEdge
7  import androidx.compose.foundation.Image
8  import androidx.compose.foundation.background
9  import androidx.compose.foundation.layout.fillMaxSize
10 import androidx.compose.foundation.layout.fillMaxWidth
11 import androidx.compose.foundation.layout.height
```

```
12              import androidx.compose.foundation.lazy.LazyColumn
13              import androidx.compose.material3.Text
14              import androidx.compose.runtime.Composable
15              import androidx.compose.ui.Modifier
16              import androidx.compose.ui.graphics.Color
17              import androidx.compose.ui.res.painterResource
18              import androidx.compose.ui.text.style.TextAlign
19              import androidx.compose.ui.tooling.preview.Preview
20              import androidx.compose.ui.unit.dp
21              import androidx.compose.ui.unit.sp
22              import com.example.columnaplication.ui.theme.
    ColumnAplicationTheme
23
24              class MainActivity : ComponentActivity() {
25                      override fun onCreate(savedInstanceState: Bundle?) {
26                              super.onCreate(savedInstanceState)
27                              enableEdgeToEdge()
28                              setContent {
29                                      ColumnAplicationTheme {
30                                              app()
31                                      }
32                              }
33                      }
34              }
35
36              @Preview(showBackground = true)
37              @Composable
38              fun app() {
39                      LazyColumn(
40                      modifier = Modifier
41                      .fillMaxSize()
42                      .background(Color.Red)
43                      ) {
44                              item {
45                                      Image(
46                                      modifier = Modifier
47                                      .fillMaxWidth()
48                                      .height(400.dp),
49                                      painter = painterResource(id = R.
    drawable.potro),
50                                      contentDescription = "Logo"
```

```
                                        )
                                }

                                item {
                                        Text(
                                        text = "AristiDevs",
                                        fontSize = 32.sp,
                                        color = Color.White,
                                        modifier = Modifier.fillMaxWidth(),
                                        textAlign = TextAlign.Center
                                        )
                                }

                                item {
                                        Text(
                                        text = "Suscribete",
                                        color = Color.White,
                                        modifier = Modifier.fillMaxWidth(),
                                        textAlign = TextAlign.Center
                                        )
                                }

                                item {
                                        Text(
                                        text = "Hola",
                                        color = Color.White,
                                        modifier = Modifier.fillMaxWidth(),
                                        textAlign = TextAlign.Center
                                        )
                                }
                        }
                }

```

7. Elementos principales utilizados

Tabla 1: Elementos principales utilizados en la práctica

Elemento	Función
`MainActivity`	Actividad principal de la aplicación.
`setContent`	Establece el contenido de la interfaz mediante Compose.
`@Composable`	Identifica una función que construye elementos de interfaz.
`@Preview`	Permite visualizar una vista previa de la interfaz.
`LazyColumn`	Organiza elementos verticalmente y permite desplazamiento.
`item{}`	Permite agregar elementos dentro de LazyColumn.
`Image`	Permite mostrar una imagen.
`Text`	Permite mostrar texto.
`Modifier`	Permite modificar características de los componentes.
`fillMaxSize()`	Permite ocupar el espacio disponible.
`fillMaxWidth()`	Permite ocupar el ancho disponible.
`height()`	Establece una altura determinada.
`background()`	Establece el color de fondo.
`TextAlign.Center`	Centra el contenido del texto.

8. Conclusiones

La realización de esta práctica permitió comprender los fundamentos de la organización de interfaces mediante Jetpack Compose.

El componente `Column` demostró ser una herramienta adecuada para organizar componentes verticalmente cuando la cantidad de contenido es limitada. Su implementación permitió resolver problemas de distribución entre elementos.

Por otra parte, `LazyColumn` permitió solucionar la necesidad de desplazamiento vertical cuando el contenido supera el espacio disponible. Esto hace que sea una alternativa apropiada para interfaces que necesitan mostrar una cantidad mayor de elementos.

Asimismo, se comprobó la importancia de los modificadores de Jetpack Compose, ya que permiten controlar características como el tamaño, el ancho, la altura, el fondo y la alineación de los componentes.

En conclusión, la práctica permitió adquirir una base para continuar desarrollando interfaces Android mediante el enfoque declarativo de Jetpack Compose y comprender cómo seleccionar diferentes estructuras de organización dependiendo de las necesidades de la interfaz.

9. Trabajo futuro

Como continuación de esta práctica se pueden implementar diferentes mejoras sobre la interfaz desarrollada. Entre ellas se encuentran:

- Incorporar una mayor cantidad de elementos.
- Agregar diferentes tipos de componentes visuales.
- Utilizar listas dinámicas.
- Incorporar interacción con el usuario.
- Implementar botones y eventos.
- Mejorar la adaptación de la interfaz a diferentes tamaños de pantalla.
- Explorar otros componentes de Jetpack Compose.

10. Anexos

10.1. Evidencias adicionales

Las ocho ilustraciones incluidas en el desarrollo del documento representan las principales etapas realizadas durante la práctica.

Las imágenes utilizadas corresponden a los archivos:

- 1.png
- 2.png
- 3.png
- 4.png
- 5.png
- 6.png
- 7.png
- 8.png

11. Referencias

- AristiDevs. (2021). *Columnas en Jetpack Compose - Curso Jetpack Compose desde cero - Tutorial Android Compose [2021]*. YouTube.
- Universidad Politécnica de Pachuca. (s. f.). *Guía de estructura y desarrollo de documentos académicos*. Material proporcionado para la asignatura.